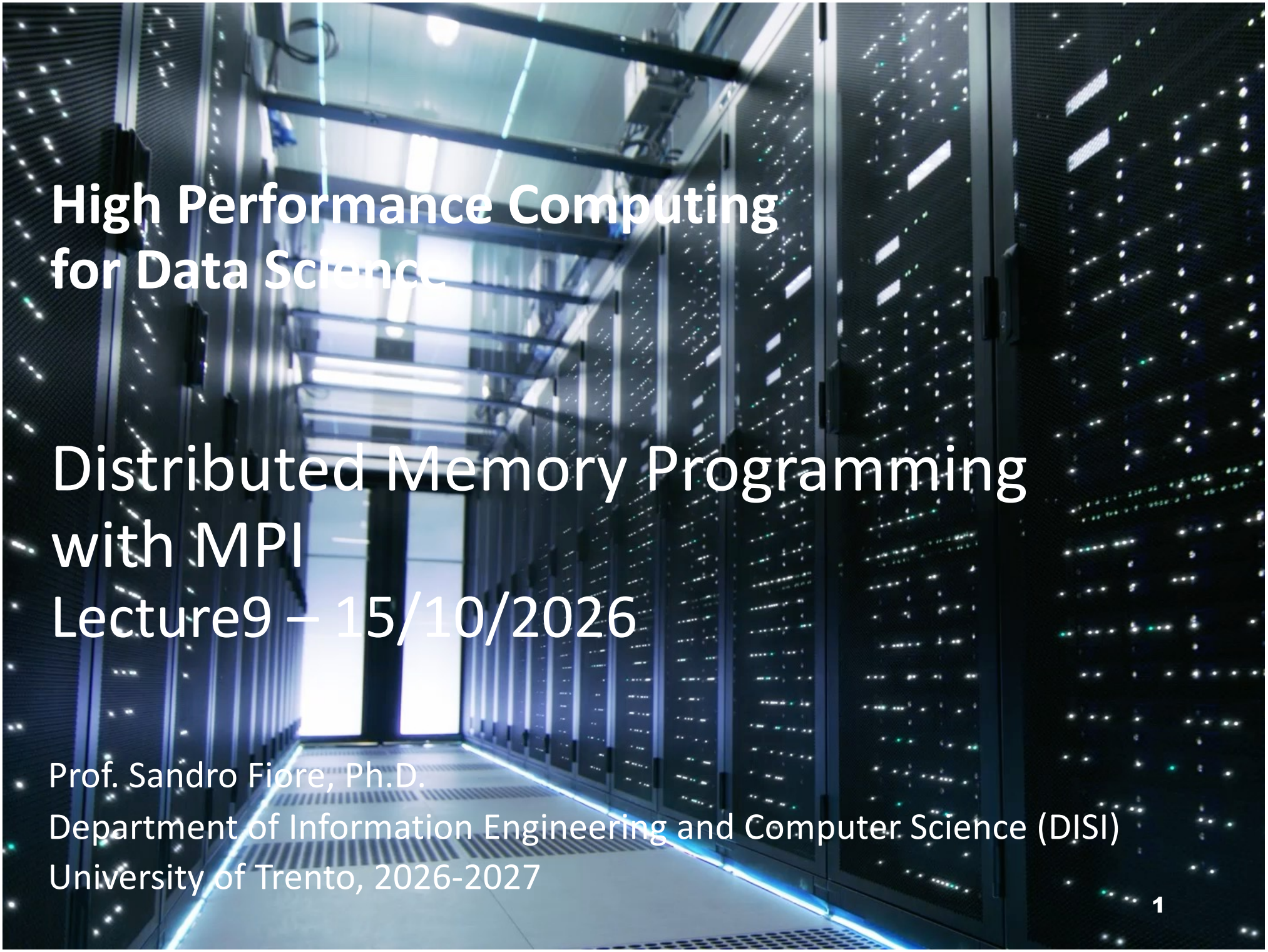


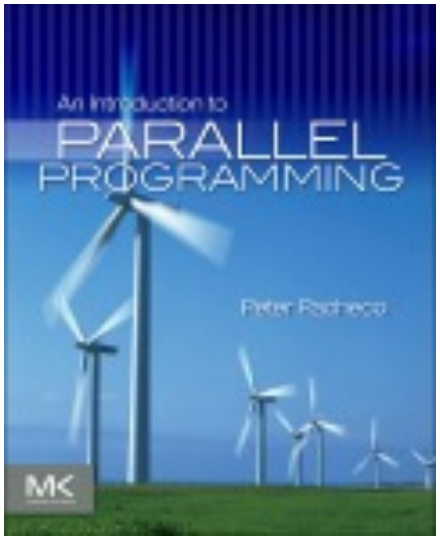
High Performance Computing for Data Science

Distributed Memory Programming with MPI
Lecture9 – 15/10/2026

Prof. Sandro Fiore, Ph.D.
Department of Information Engineering and Computer Science (DISI)
University of Trento, 2026-2027

Today's outline and material

- Hands-on reminder
- Trapezoidal rule (second part)
- Collective communications
- Data Distribution



Today's material

Book
An Introduction to Parallel Programming by Peter Pacheco

Chapter 3
Distributed Memory Programming with MPI

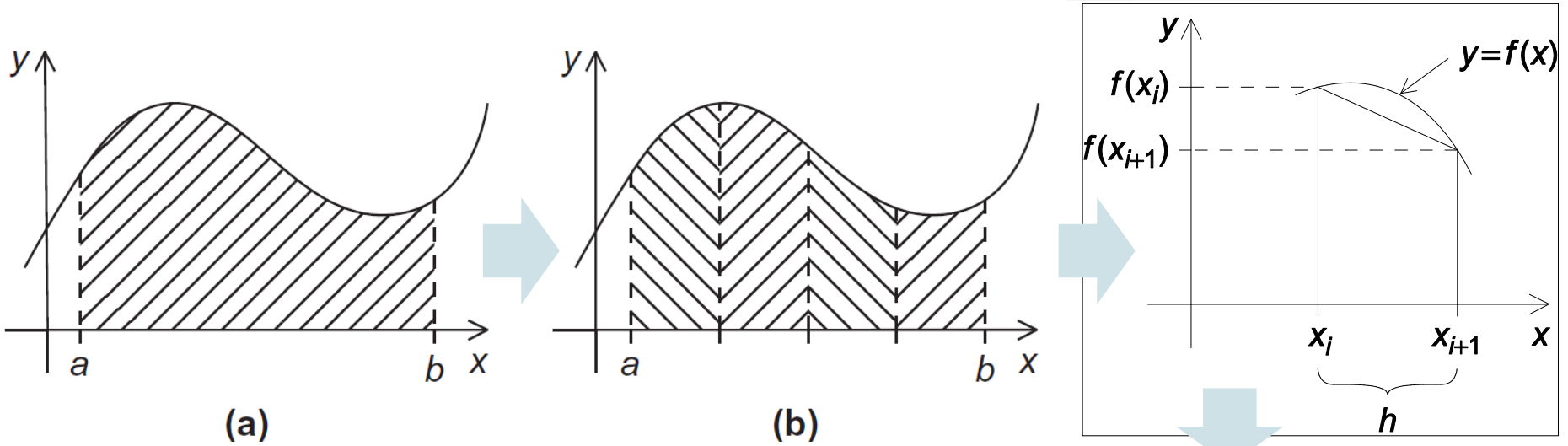
High Performance Computing for Data Science

MPI Communication
Lecture9 – 15/10/2026

The Trapezoidal rule in MPI

Prof. Sandro Fiore, Ph.D.
Department of Information Engineering and Computer Science (DISI)
University of Trento, 2026-2027

The Trapezoidal Rule



$$\text{Sum of trapezoid areas} = h[f(x_0)/2 + f(x_1) + f(x_2) + \cdots + f(x_{n-1}) + f(x_n)/2]$$

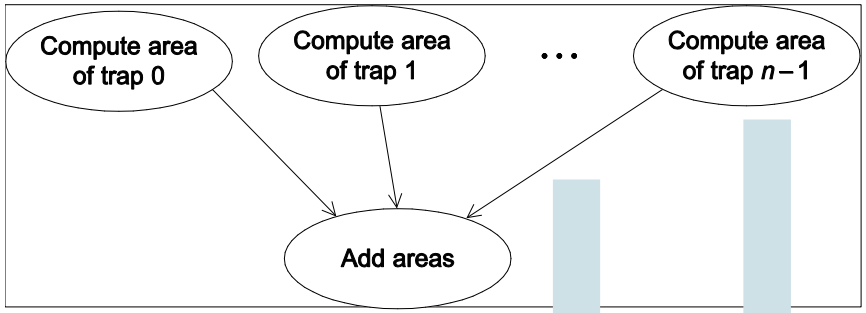
```
/* Input:  a, b, n */
h = (b-a)/n;
approx = (f(a) + f(b))/2.0;
for (i = 1; i <= n-1; i++) {
   x_i = a + i*h;
   approx += f(x_i);
}
approx = h*approx;
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

Parallelizing the Trapezoidal Rule

1. Partition problem solution into tasks.
2. Identify communication channels between tasks.
3. Aggregate tasks into composite tasks.
4. Map composite tasks to cores.




```
1    Get a, b, n;
2    h = (b–a)/n;
3    local_n = n/comm_sz;
4    local_a = a + my_rank*local_n;
5    local_b = local_a + local_n*h;
6    local_integral = Trap(local_a, local_b, local_n, h);
7    if (my_rank != 0)
8       Send local_integral to process 0;
9    else /* my_rank == 0 */
10      total_integral = local_integral;
11      for (proc = 1; proc < comm_sz; proc++) {
12         Receive local_integral from proc;
13         total_integral += local_integral;
14      }
15   }
16   if (my_rank == 0)
17      print result;
```

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

First version

```
int main(void) {
   int my_rank, comm_sz, n = 1024, local_n;
   double a = 0.0, b = 3.0, h, local_a, local_b;
   double local_int, total_int;
   int source;

   MPI_Init(NULL, NULL);
   MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
   MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);

   h = (b-a)/n;          /* h is the same for all processes */
   local_n = n/comm_sz;  /* So is the number of trapezoids  */

   local_a = a + my_rank*local_n*h;
   local_b = local_a + local_n*h;
   local_int = Trap(local_a, local_b, local_n, h);

   if (my_rank != 0) {
      MPI_Send(&local_int, 1, MPI_DOUBLE, 0, 0,
            MPI_COMM_WORLD);
```

```
   } else {
      total_int = local_int;
      for (source = 1; source < comm_sz; source++) {
         MPI_Recv(&local_int, 1, MPI_DOUBLE, source, 0,
               MPI_COMM_WORLD, MPI_STATUS_IGNORE);
         total_int += local_int;
      }
   }

   if (my_rank == 0) {
      printf("With n = %d trapezoids, our estimate\n", n);
      printf("of the integral from %f to %f = %.15e\n",
          a, b, total_int);
   }
   MPI_Finalize();
   return 0;
} /*  main  */
```

```
/* Input:  a, b, n */
h = (b-a)/n;
approx = (f(a) + f(b))/2.0;
for (i = 1; i <= n-1; i++) {
   x_i = a + i*h;
   approx += f(x_i);
}
approx = h*approx;
```

```
double Trap(
      double left_endpt  /* in */,
      double right_endpt /* in */,
      int    trap_count  /* in */,
      double base_len    /* in */) {
   double estimate, x;
   int i;

   estimate = (f(left_endpt) + f(right_endpt))/2.0;
   for (i = 1; i <= trap_count-1; i++) {
      x = left_endpt + i*base_len;
      estimate += f(x);
   }
   estimate = estimate*base_len;

   return estimate;
} /*  Trap  */
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

Input

- Most MPI implementations only allow process 0 in MPI_COMM_WORLD access to stdin.
- Process 0 must read the data (scanf) and send to the other processes.

```
. . .
MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);

Get_data(my_rank, comm_sz, &a, &b, &n);

h = (b-a)/n;
. . .
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

Function for reading user input

```
void Get_input(
      int       my_rank   /* in  */,
      int       comm_sz   /* in  */,
      double*   a_p       /* out */,
      double*   b_p       /* out */,
      int*      n_p       /* out */) {
   int dest;

   if (my_rank == 0) {
      printf("Enter a, b, and n\n");
      scanf("%lf %lf %d", a_p, b_p, n_p);
      for (dest = 1; dest < comm_sz; dest++) {
         MPI_Send(a_p, 1, MPI_DOUBLE, dest, 0, MPI_COMM_WORLD);
         MPI_Send(b_p, 1, MPI_DOUBLE, dest, 0, MPI_COMM_WORLD);
         MPI_Send(n_p, 1, MPI_INT, dest, 0, MPI_COMM_WORLD);
      }
   } else { /* my_rank != 0 */
      MPI_Recv(a_p, 1, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD,
            MPI_STATUS_IGNORE);
      MPI_Recv(b_p, 1, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD,
            MPI_STATUS_IGNORE);
      MPI_Recv(n_p, 1, MPI_INT, 0, 0, MPI_COMM_WORLD,
            MPI_STATUS_IGNORE);
   }
}  /* Get_input */
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

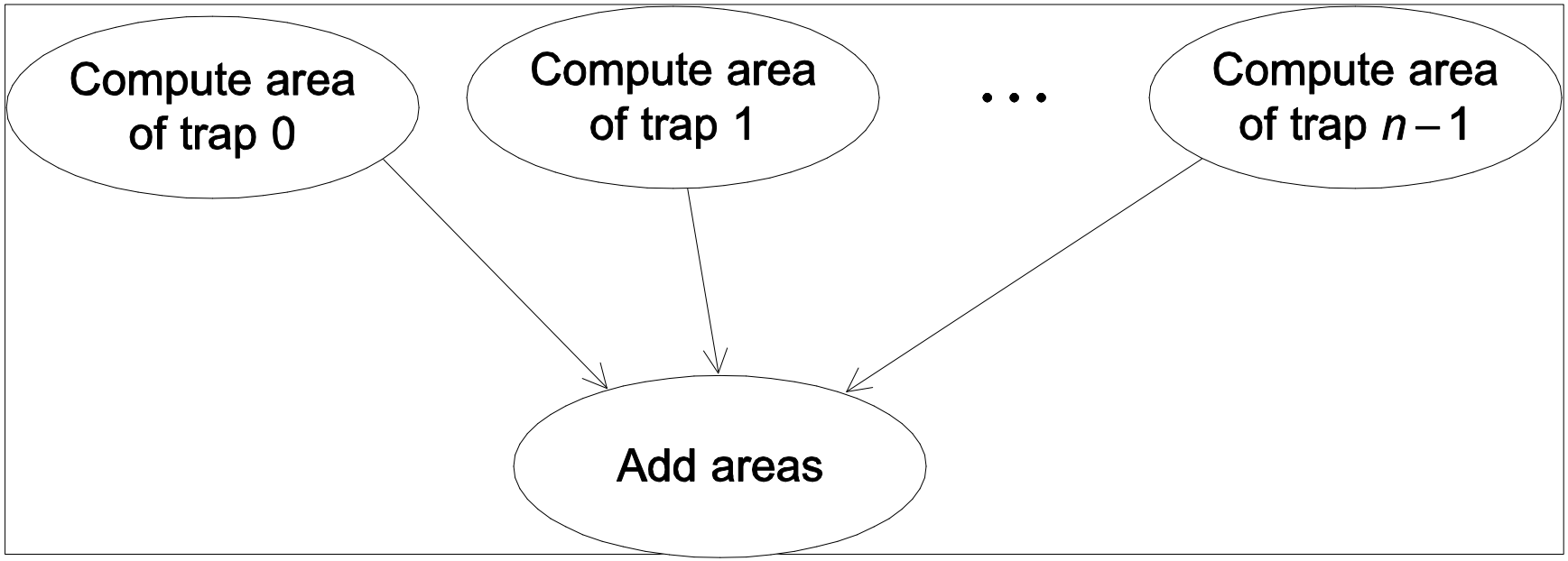
High Performance Computing for Data Science

MPI Communication
Lecture9 - 16/10/2025

Collective communications

Prof. Sandro Fiore, Ph.D.
Department of Information Engineering and Computer Science (DISI)
University of Trento, 2025-2026

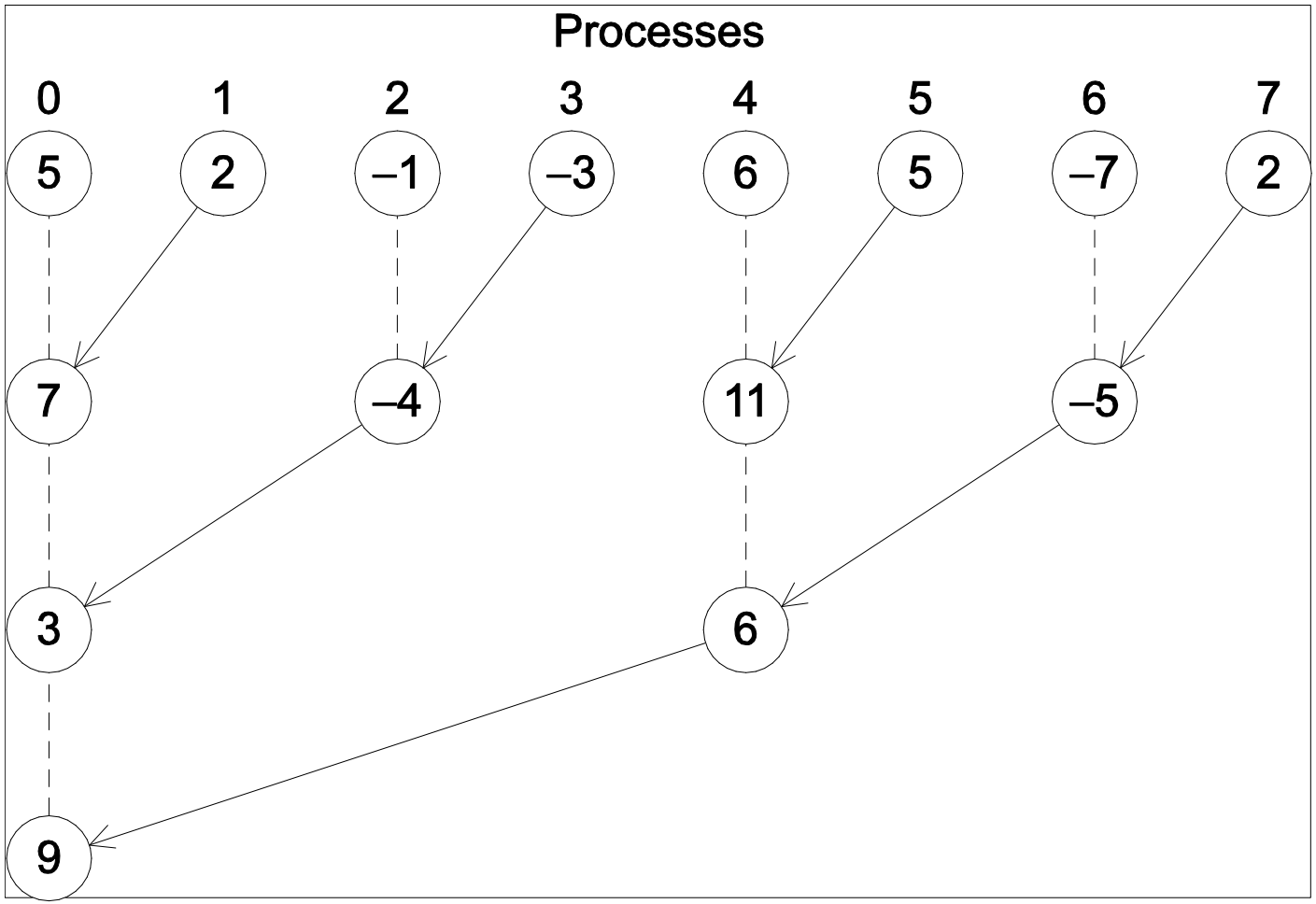
Tasks and communications for Trapezoidal Rule



Too much burden on the collector node
Can we better split the communication?

Copyright © 2010, Elsevier Inc. All rights Reserved

A tree-structured global sum



MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

Tree-structured communication

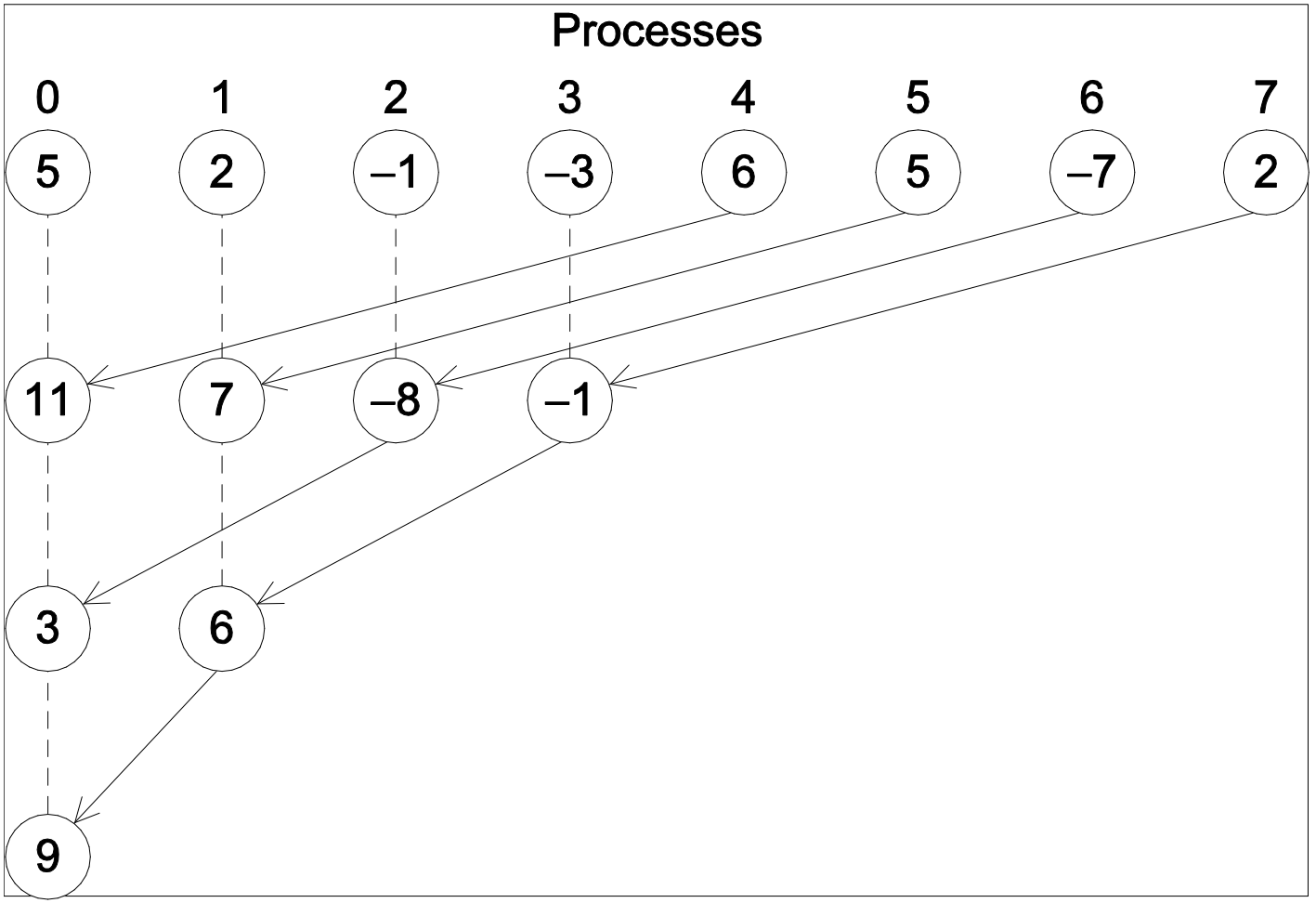
1. In the first phase:
   (a) Process 1 sends to 0, 3 sends to 2, 5 sends to 4, and 7 sends to 6.
   (b) Processes 0, 2, 4, and 6 add in the received values.
   (c) Processes 2 and 6 send their new values to processes 0 and 4, respectively.
   (d) Processes 0 and 4 add the received values into their new values.

2. (a) Process 4 sends its newest value to process 0.
   (b) Process 0 adds the received value to its newest value.

Is that the only way?

MK MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

An alternative tree-structured global sum



MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

MPI_Reduce

```
int MPI_Reduce(
      void*         input_data_p   /* in  */,
      void*         output_data_p  /* out */,
      int           count          /* in  */,
      MPI_Datatype  datatype       /* in  */,
      MPI_Op        operator       /* in  */,
      int           dest_process   /* in  */,
      MPI_Comm      comm           /* in  */);
```

```
MPI_Reduce(&local_int, &total_int, 1, MPI_DOUBLE, MPI_SUM, 0,
      MPI_COMM_WORLD);
```

```
double local_x[N], sum[N];
. . .
MPI_Reduce(local_x, sum, N, MPI_DOUBLE, MPI_SUM, 0,
      MPI_COMM_WORLD);
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

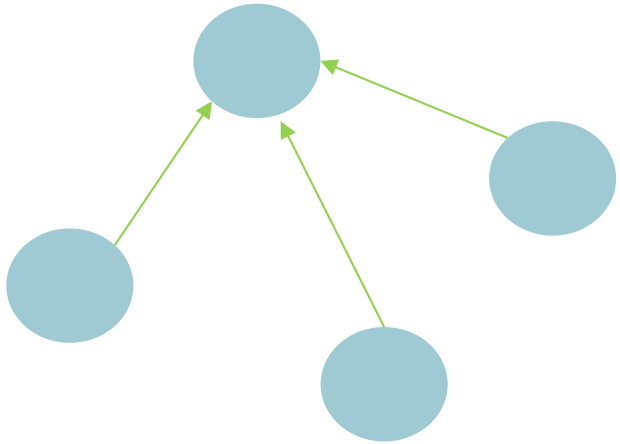
Predefined reduction operators in MPI

Operation Value	Meaning
MPI_MAX	Maximum
MPI_MIN	Minimum
MPI_SUM	Sum
MPI_PROD	Product
MPI_LAND	Logical and
MPI_BAND	Bitwise and
MPI_LOR	Logical or
MPI_BOR	Bitwise or
MPI_LXOR	Logical exclusive or
MPI_BXOR	Bitwise exclusive or
MPI_MAXLOC	Maximum and location of maximum
MPI_MINLOC	Minimum and location of minimum

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

Communication hands-on

Implement a “reduce” by using send and receive + average/max/min over the set of data collected



Copyright © 2010, Elsevier Inc. All rights Reserved

Collective vs. Point-to-Point Communications

- All the processes in the communicator must call the same collective function.

- For example, a program that attempts to match a call to MPI_Reduce on one process with a call to MPI_Recv on another process is erroneous, and, in all likelihood, the program will hang or crash.

Copyright © 2010, Elsevier Inc. All rights Reserved

Collective vs. Point-to-Point Communications

- The arguments passed by each process to an MPI collective communication must be "compatible."

- For example, if one process passes in 0 as the dest_process and another passes in 1, then the outcome of a call to MPI_Reduce is erroneous, and, once again, the program is likely to hang or crash.

Copyright © 2010, Elsevier Inc. All rights Reserved

Collective vs. Point-to-Point Communications

- The output_data_p argument is only used on dest_process

- However, all of the processes still need to pass in an actual argument corresponding to output_data_p, even if it's just NULL.

Copyright © 2010, Elsevier Inc. All rights Reserved

Collective vs. Point-to-Point Communications

- **Point-to-point** communications are matched on the basis of **tags** and **communicators**.
- **Collective** communications **don't use tags**.
- They're matched solely on the basis of the **communicator** and the **order** in which they're called.

Copyright © 2010, Elsevier Inc. All rights Reserved

Example (1)

Time	Process 0	Process 1	Process 2
0	`a = 1; c = 2`	`a = 1; c = 2`	`a = 1; c = 2`
1	`MPI_Reduce(&a, &b, ...)`	`MPI_Reduce(&c, &d, ...)`	`MPI_Reduce(&a, &b, ...)`
2	`MPI_Reduce(&c, &d, ...)`	`MPI_Reduce(&a, &b, ...)`	`MPI_Reduce(&c, &d, ...)`

Multiple calls to MPI_Reduce

Process 0 is acting as destination

What are the values in b and d for process 0?

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

Example (2)

- Suppose that each process calls MPI_Reduce with operator MPI_SUM, and destination process 0.

- At first glance, it might seem that after the two calls to MPI_Reduce, the value of b will be 3, and the value of d will be 6.

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

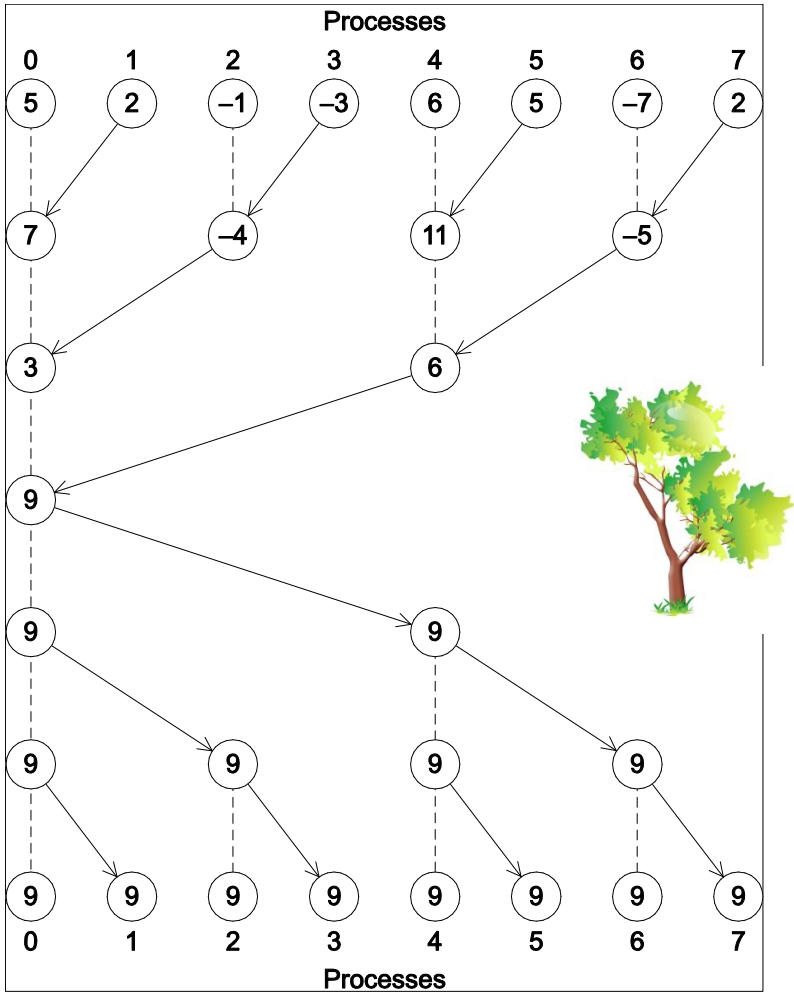
MPI_Allreduce

- Useful in a situation in which all of the processes need the result of a global sum in order to complete some larger computation.

```
int MPI_Allreduce(
        void*        input_data_p    /* in  */,
        void*        output_data_p   /* out */,
        int          count           /* in  */,
        MPI_Datatype datatype        /* in  */,
        MPI_Op       operator        /* in  */,
        MPI_Comm     comm            /* in  */);
```

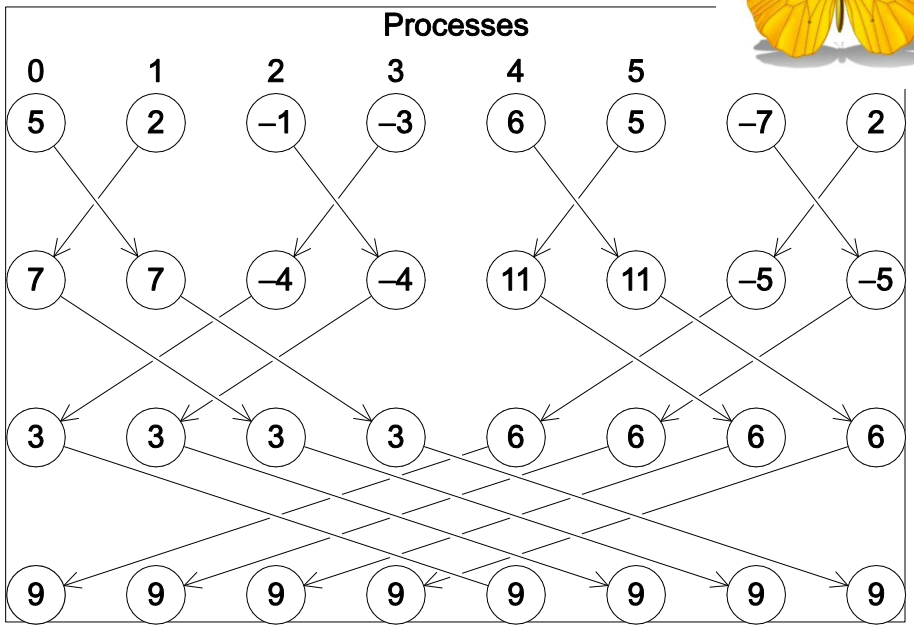
MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

MPI_Allreduce – two implementations









“All” = “+ broadcast”

Tree structured and buttefly

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

Broadcast

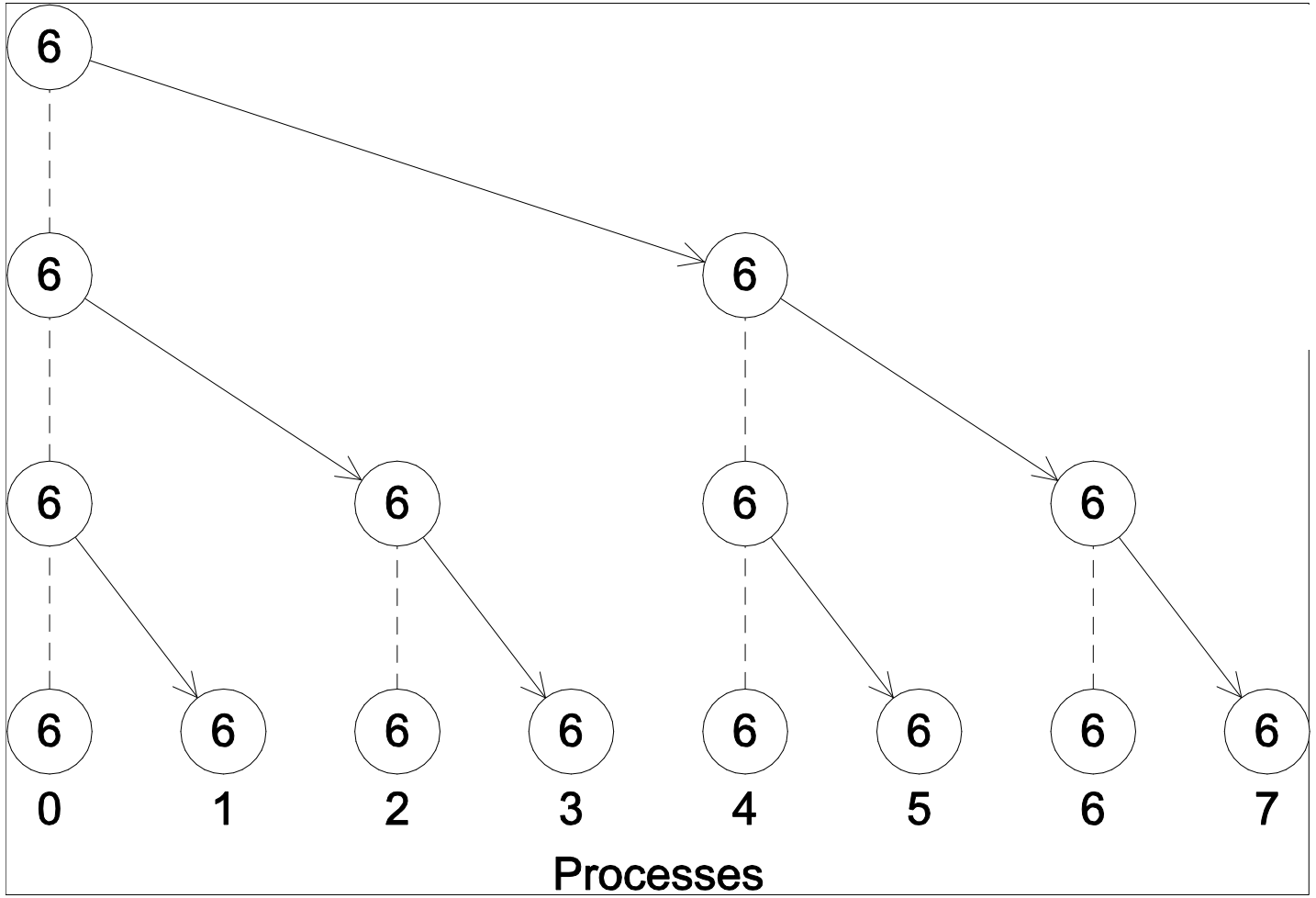
- Data belonging to a single process is sent to all of the processes in the communicator.

```
int MPI_Bcast(
      void*          data_p       /* in/out */,
      int            count        /* in     */,
      MPI_Datatype   datatype     /* in     */,
      int            source_proc  /* in     */,
      MPI_Comm       comm         /* in     */);
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

Broadcast implementation





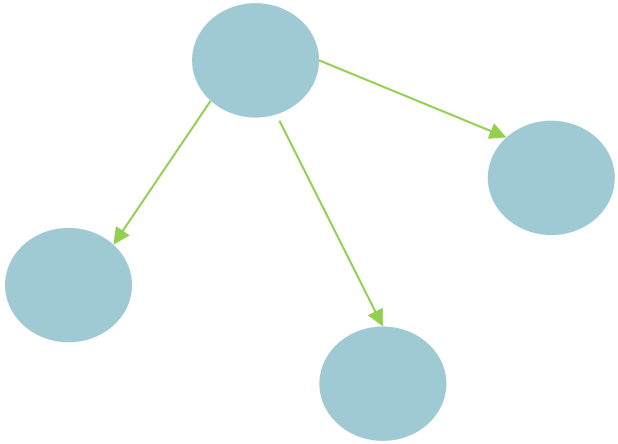
Tree structured implementation

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

Get_input without MPI_Bcast

```
void Get_input(
      int       my_rank   /* in  */,
      int       comm_sz   /* in  */,
      double*   a_p       /* out */,
      double*   b_p       /* out */,
      int*      n_p       /* out */) {
   int dest;

   if (my_rank == 0) {
      printf("Enter a, b, and n\n");
      scanf("%lf %lf %d", a_p, b_p, n_p);
      for (dest = 1; dest < comm_sz; dest++) {
         MPI_Send(a_p, 1, MPI_DOUBLE, dest, 0, MPI_COMM_WORLD);
         MPI_Send(b_p, 1, MPI_DOUBLE, dest, 0, MPI_COMM_WORLD);
         MPI_Send(n_p, 1, MPI_INT, dest, 0, MPI_COMM_WORLD);
      }
   } else { /* my_rank != 0 */
      MPI_Recv(a_p, 1, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD,
            MPI_STATUS_IGNORE);
      MPI_Recv(b_p, 1, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD,
            MPI_STATUS_IGNORE);
      MPI_Recv(n_p, 1, MPI_INT, 0, 0, MPI_COMM_WORLD,
            MPI_STATUS_IGNORE);
   }
}  /* Get_input */
```



Too expensive in terms of communication calls

Copyright © 2010, Elsevier Inc. All rights Reserved

Get_input that uses MPI_Bcast

```
void Get_input(
      int       my_rank   /* in  */,
      int       comm_sz   /* in  */,
      double*   a_p       /* out */,
      double*   b_p       /* out */,
      int*      n_p       /* out */) {

   if (my_rank == 0) {
      printf("Enter a, b, and n\n");
      scanf("%lf %lf %d", a_p, b_p, n_p);
   }
   MPI_Bcast(a_p, 1, MPI_DOUBLE, 0, MPI_COMM_WORLD);
   MPI_Bcast(b_p, 1, MPI_DOUBLE, 0, MPI_COMM_WORLD);
   MPI_Bcast(n_p, 1, MPI_INT, 0, MPI_COMM_WORLD);
}  /* Get_input */
```

All processes must run the collective

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

Ring solution (open ring, broadcast-like)

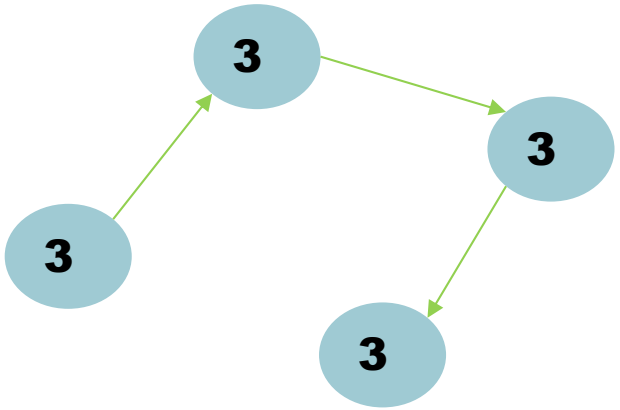
```
#include <stdio.h>
#include "mpi.h"

int main( argc, argv )
int argc;
char **argv;
{
    int rank, value, size;
    MPI_Status status;

    MPI_Init( &argc, &argv );

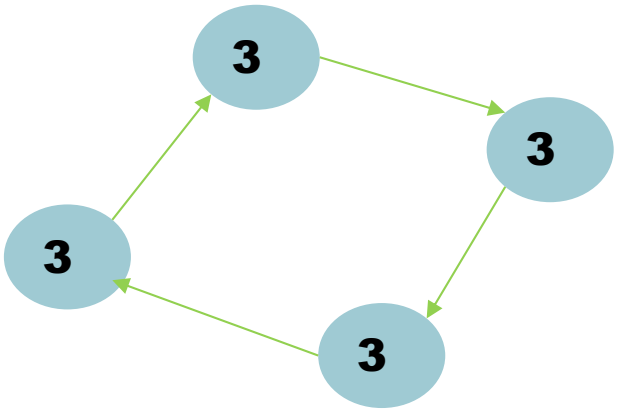
    MPI_Comm_rank( MPI_COMM_WORLD, &rank );
    MPI_Comm_size( MPI_COMM_WORLD, &size );
    do {
        if (rank == 0) {
            scanf( "%d", &value );
            MPI_Send( &value, 1, MPI_INT, rank + 1, 0, MPI_COMM_WORLD );
        }
        else {
            MPI_Recv( &value, 1, MPI_INT, rank - 1, 0, MPI_COMM_WORLD,
                      &status );
            if (rank < size - 1)
                MPI_Send( &value, 1, MPI_INT, rank + 1, 0, MPI_COMM_WORLD );
        }
        printf( "Process %d got %d\n", rank, value );
    } while (value >= 0);

    MPI_Finalize( );
    return 0;
}
```



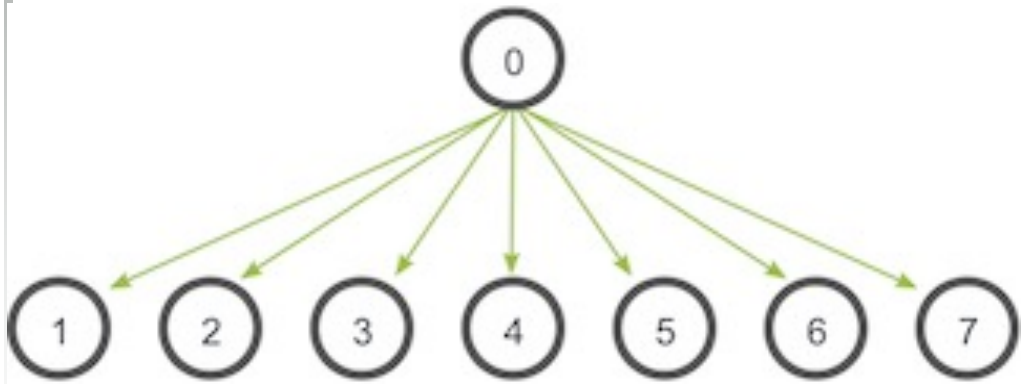
MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

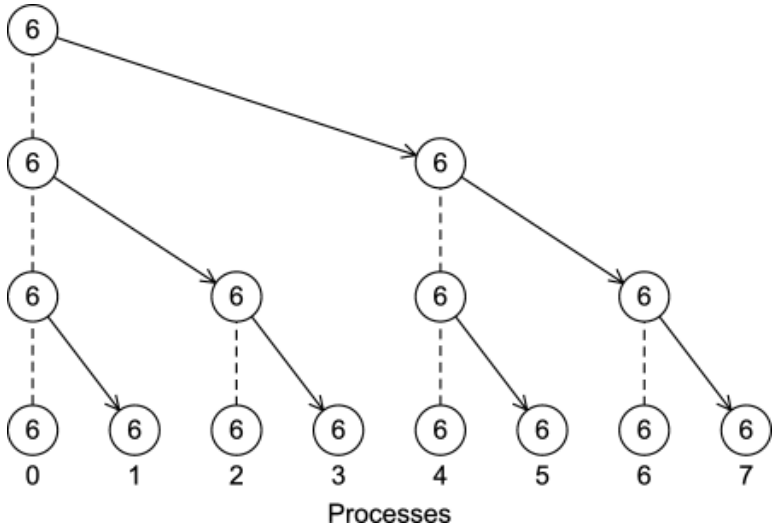
Closed ring exercise (solution)



Copyright © 2010, Elsevier Inc. All rights Reserved

Broadcast tree-struct “challenge”






```
int MPI_Bcast(
      void*         data_p      /* in/out */,
      int           count       /* in     */,
      MPI_Datatype  datatype    /* in     */,
      int           source_proc /* in     */,
      MPI_Comm      comm        /* in     */);
```

We could implement our “myMPI_Bcast” by using send and receive only

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

High Performance Computing for Data Science

MPI Communication
Lecture9 - 15/10/2026

Data distribution: Scatter and Gather

Prof. Sandro Fiore, Ph.D.
Department of Information Engineering and Computer Science (DISI)
University of Trento, 2026-2027

Data distributions (vector sum)

$$
\begin{aligned}
\mathbf{x}+\mathbf{y} &= (x_0, x_1, \ldots, x_{n-1}) + (y_0, y_1, \ldots, y_{n-1}) \\
&= (x_0 + y_0, x_1 + y_1, \ldots, x_{n-1} + y_{n-1}) \\
&= (z_0, z_1, \ldots, z_{n-1}) \\
&= \mathbf{z}
\end{aligned}
$$

```
void Vector_sum(double x[], double y[], double z[], int n) {
   int i;

   for (i = 0; i < n; i++)
      z[i] = x[i] + y[i];
}  /* Vector_sum */
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

Parallel implementation of vector addition

```
void Parallel_vector_sum(
      double  local_x[]  /* in  */,
      double  local_y[]  /* in  */,
      double  local_z[]  /* out */,
      int     local_n    /* in  */) {
   int local_i;

   for (local_i = 0; local_i < local_n; local_i++)
      local_z[local_i] = local_x[local_i] + local_y[local_i];
}  /* Parallel_vector_sum */
```

It's actually still a serial version!

MK MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

Scatter

- MPI_Scatter can be used in a function that reads in an entire vector on process 0 but only sends the needed components to each of the other processes.

```
int MPI_Scatter(
      void*          send_buf_p  /* in  */,
      int            send_count  /* in  */,
      MPI_Datatype   send_type   /* in  */,
      void*          recv_buf_p  /* out */,
      int            recv_count  /* in  */,
      MPI_Datatype   recv_type   /* in  */,
      int            src_proc    /* in  */,
      MPI_Comm       comm        /* in  */);
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

Reading and distributing a vector

```
void Read_vector(
      double     local_a[]   /* out */,
      int        local_n     /* in  */,
      int        n           /* in  */,
      char       vec_name[]  /* in  */,
      int        my_rank     /* in  */,
      MPI_Comm   comm        /* in  */) {

   double* a = NULL;
   int i;

   if (my_rank == 0) {
      a = malloc(n*sizeof(double));
      printf("Enter the vector %s\n", vec_name);
      for (i = 0; i < n; i++)
         scanf("%lf", &a[i]);
      MPI_Scatter(a, local_n, MPI_DOUBLE, local_a, local_n, MPI_DOUBLE,
            0, comm);
      free(a);
   } else {
      MPI_Scatter(a, local_n, MPI_DOUBLE, local_a, local_n, MPI_DOUBLE,
            0, comm);
   }
} /* Read_vector */
```

```
int MPI_Scatter(
      void*         send_buf_p  /* in  */,
      int           send_count  /* in  */,
      MPI_Datatype  send_type   /* in  */,
      void*         recv_buf_p  /* out */,
      int           recv_count  /* in  */,
      MPI_Datatype  recv_type   /* in  */,
      int           src_proc    /* in  */,
      MPI_Comm      comm        /* in  */);
```


Copyright © 2010, Elsevier Inc. All rights Reserved

Reading and distributing a vector

```
void Read_vector(
      double     local_a[]   /* out */,
      int        local_n     /* in  */,
      int        n           /* in  */,
      char       vec_name[]  /* in  */,
      int        my_rank     /* in  */,
      MPI_Comm   comm        /* in  */) {

   double* a = NULL;
   int i;

   if (my_rank == 0) {
      a = malloc(n*sizeof(double));
      printf("Enter the vector %s\n", vec_name);
      for (i = 0; i < n; i++)
         scanf("%lf", &a[i]);
      MPI_Scatter(a, local_n, MPI_DOUBLE, local_a, local_n, MPI_DOUBLE,
            0, comm);
      free(a);
   } else {
      MPI_Scatter(a, local_n, MPI_DOUBLE, local_a, local_n, MPI_DOUBLE,
            0, comm);
   }
} /* Read_vector */
```

?

```
int MPI_Scatter(
      void*         send_buf_p  /* in  */,
      int           send_count  /* in  */,
      MPI_Datatype  send_type   /* in  */,
      void*         recv_buf_p  /* out */,
      int           recv_count  /* in  */,
      MPI_Datatype  recv_type   /* in  */,
      int           src_proc    /* in  */,
      MPI_Comm      comm        /* in  */);
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

Gather

- Collect all of the components of the vector onto process *dest_proc*, and then process *dest_proc* can process all of the components.

```
int MPI_Gather(
      void*          send_buf_p   /* in  */,
      int            send_count   /* in  */,
      MPI_Datatype   send_type    /* in  */,
      void*          recv_buf_p   /* out */,
      int            recv_count   /* in  */,
      MPI_Datatype   recv_type    /* in  */,
      int            dest_proc    /* in  */,
      MPI_Comm       comm         /* in  */);
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

Print a distributed vector

```
void Print_vector(
      double     local_b[]  /* in */,
      int        local_n    /* in */,
      int        n          /* in */,
      char       title[]    /* in */,
      int        my_rank    /* in */,
      MPI_Comm   comm       /* in */) {

   double* b = NULL;
   int i;

   if (my_rank == 0) {
      b = malloc(n*sizeof(double));
      MPI_Gather(local_b, local_n, MPI_DOUBLE, b, local_n, MPI_DOUBLE,
            0, comm);
      printf("%s\n", title);
      for (i = 0; i < n; i++)
         printf("%f ", b[i]);
      printf("\n");
      free(b);
   } else {
      MPI_Gather(local_b, local_n, MPI_DOUBLE, b, local_n, MPI_DOUBLE,
            0, comm);
   }
}  /* Print_vector */
```

```
int MPI_Gather(
      void*         send_buf_p  /* in  */,
      int           send_count  /* in  */,
      MPI_Datatype  send_type   /* in  */,
      void*         recv_buf_p  /* out */,
      int           recv_count  /* in  */,
      MPI_Datatype  recv_type   /* in  */,
      int           dest_proc   /* in  */,
      MPI_Comm      comm        /* in  */);
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

Print a distributed vector

```
void Print_vector(
      double     local_b[]  /* in */,
      int        local_n    /* in */,
      int        n          /* in */,
      char       title[]    /* in */,
      int        my_rank    /* in */,
      MPI_Comm   comm       /* in */) {

   double* b = NULL;
   int i;

   if (my_rank == 0) {
      b = malloc(n*sizeof(double));
      MPI_Gather(local_b, local_n, MPI_DOUBLE, b, local_n, MPI_DOUBLE,
            0, comm);
      printf("%s\n", title);
      for (i = 0; i < n; i++)
         printf("%f ", b[i]);
      printf("\n");
      free(b);
   } else {
      MPI_Gather(local_b, local_n, MPI_DOUBLE, b, local_n, MPI_DOUBLE,
            0, comm);
   }
}  /* Print_vector */
```

```
int MPI_Gather(
      void*         send_buf_p  /* in  */,
      int           send_count  /* in  */,
      MPI_Datatype  send_type   /* in  */,
      void*         recv_buf_p  /* out */,
      int           recv_count  /* in  */,
      MPI_Datatype  recv_type   /* in  */,
      int           dest_proc   /* in  */,
      MPI_Comm      comm        /* in  */);
```

?

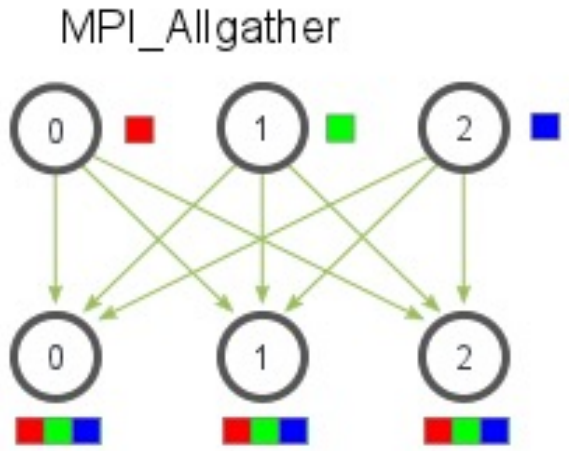
MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

Allgather

- Concatenates the contents of each process' send_buf_p and stores this in each process' recv_buf_p.
- As usual, recv_count is the amount of data being received from each process.

```
int MPI_Allgather(
      void*          send_buf_p   /* in  */,
      int            send_count   /* in  */,
      MPI_Datatype   send_type    /* in  */,
      void*          recv_buf_p   /* out */,
      int            recv_count   /* in  */,
      MPI_Datatype   recv_type    /* in  */,
      MPI_Comm       comm         /* in  */);
```



MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

Matrix-vector multiplication

$A = (a_{ij})$ is an $m \times n$ matrix

$\mathbf{x}$ is a vector with n components

$\mathbf{y} = A\mathbf{x}$ is a vector with m components

$$y_i = a_{i0}x_0 + a_{i1}x_1 + a_{i2}x_2 + \cdots a_{i,n-1}x_{n-1}$$

i-th component of y

Dot product of the ith row of A with x.

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

Multiply a matrix by a vector

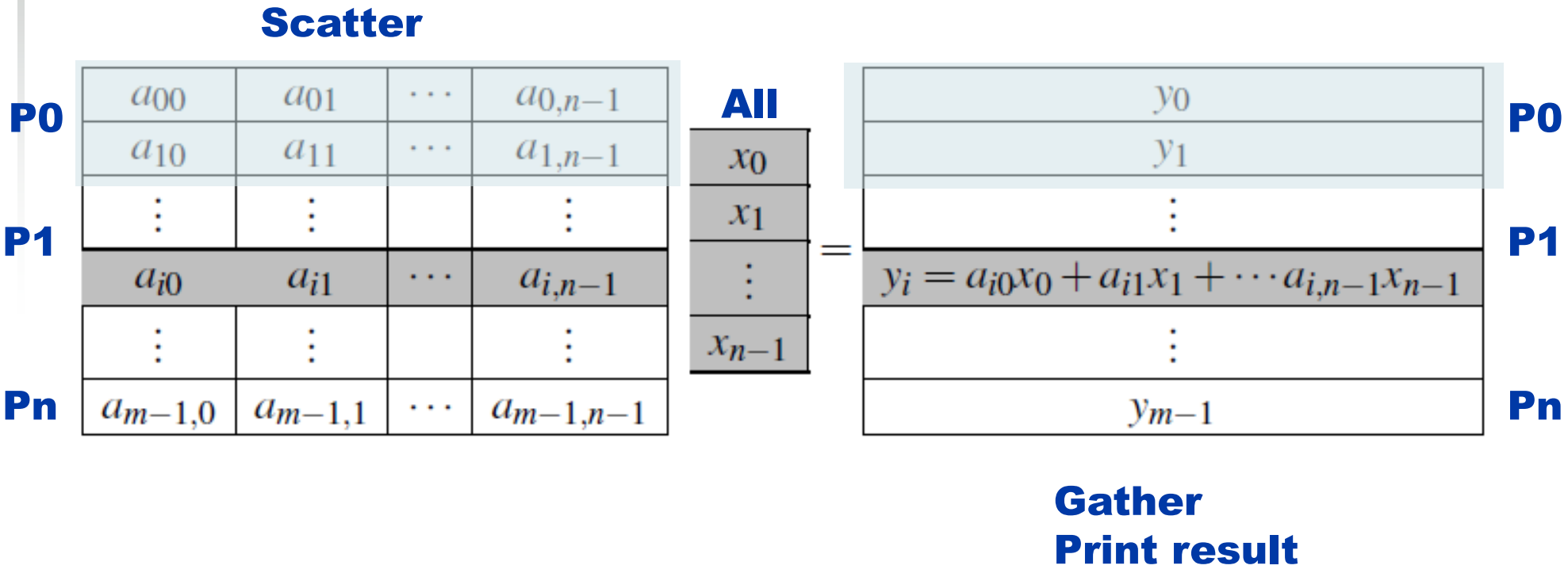
```
/* For each row of A */
for (i = 0; i < m; i++) {
    /* Form dot product of ith row with x */
    y[i] = 0.0;

    for (j = 0; j < n; j++)
        y[i] += A[i][j]*x[j];
}
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

Matrix-vector multiplication



MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

An MPI matrix-vector multiplication function

```
void Mat_vect_mult(
      double     local_A[]  /* in  */,
      double     local_x[]  /* in  */,
      double     local_y[]  /* out */,
      int        local_m    /* in  */,
      int        n          /* in  */,
      int        local_n    /* in  */,
      MPI_Comm   comm       /* in  */) {
   double* x;
   int local_i, j;
   int local_ok = 1;

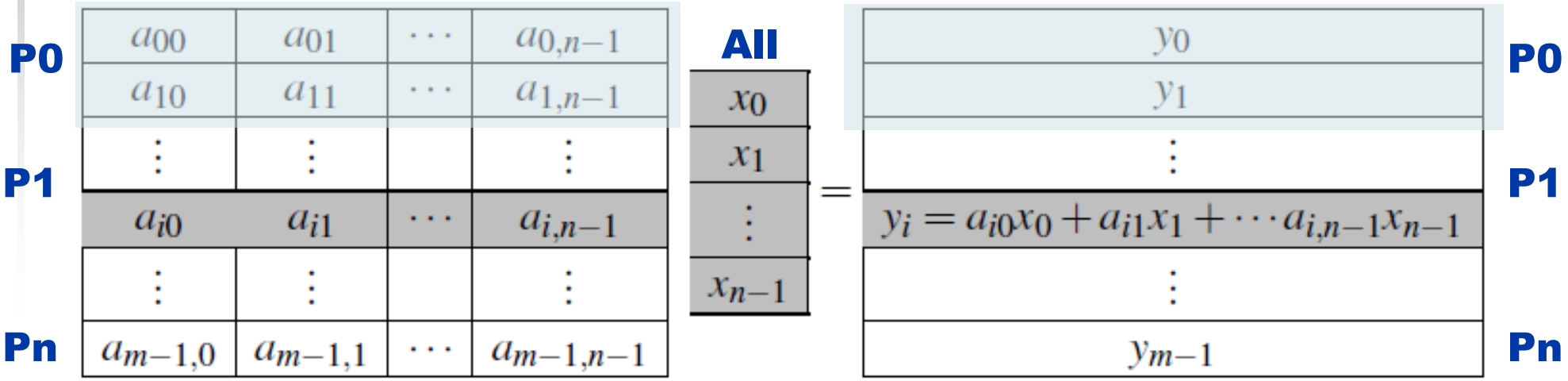
   x = malloc(n*sizeof(double));
   MPI_Allgather(local_x, local_n, MPI_DOUBLE,
         x, local_n, MPI_DOUBLE, comm);

   for (local_i = 0; local_i < local_m; local_i++) {
      local_y[local_i] = 0.0;
      for (j = 0; j < n; j++)
         local_y[local_i] += local_A[local_i*n+j]*x[j];
   }
   free(x);
}  /* Mat_vect_mult */
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

Matrix-vector multiplication



$$\begin{array}{|cccc|} a_{00} & a_{01} & \cdots & a_{0,n-1} \\ a_{10} & a_{11} & \cdots & a_{1,n-1} \\ \vdots & \vdots & & \vdots \\ a_{i0} & a_{i1} & \cdots & a_{i,n-1} \\ \vdots & \vdots & & \vdots \\ a_{m-1,0} & a_{m-1,1} & \cdots & a_{m-1,n-1} \end{array} \quad \begin{array}{|c|} x_0 \\ x_1 \\ \vdots \\ x_{n-1} \end{array} \quad = \quad \begin{array}{|c|} y_0 \\ y_1 \\ \vdots \\ y_i = a_{i0}x_0 + a_{i1}x_1 + \cdots a_{i,n-1}x_{n-1} \\ \vdots \\ y_{m-1} \end{array}$$

AllGather
Loop
Print result

Feedback loop

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

Question time

Feedback form

- Throughout the entire HPC4DS course there will be a form available for your feedback
  - Please provide any comment about:
    - Pros
    - Cons
    - Aspects that were not clear enough during the class
    - Any other feedback you think can be relevant for the course
    - …
- Information are gathered in an anonymous way

https://docs.google.com/forms/d/e/1FAIpQLScSVpNPfMP8poNNz-nLl9xlKlHkeBM2x_JgWyA0qQu6mOBl5A/viewform